Influencia de los Algoritmos Genéticos en la generación de horarios en la Unidad Educativa Ana Luisa Leoro

Influence of genetic algorithms in the generation of time in the Ana Luisa Leoro Educational Unit

Edgar Javier Terán Pozo

edgarjtp@gmail.com

Facultad de Sistemas Mercantiles, Universidad Regional Autónoma de los Andes

Transversal Central, Ibarra – Ecuador

Resumen: *En este artículo se demuestra la conveniencia de la aplicación de un algoritmo genético como apoyo para la generación de horarios académicos con la finalidad de ofrecer una opción de optimización del proceso para la asignación de docentes de la Unidad Educativa Ana Luisa Leoro, institución educativa con mayor envergadura en cantidad de alumnos, docentes e infraestructura. El método de investigación fue mixta cuantitativa – cualitativa donde se analizaron funciones para la optimización de recursos como tiempo, disponibilidad y accesibilidad a la información de los horarios; a través de la entrevista, se recogieron los datos precisos para realizar una investigación aplicada con apoyo de la metodología XP con sus 4 fases: (G1) planeación, (G2) diseño, (G3) codificación y (G4) pruebas, enfocándose en acelerar el proceso de resolución de problemas de gran tamaño. Se realizó el modelado de la solución y la implementación del algoritmo propuesto en el lenguaje de programación Python con el IDE JetBrains PyCharm acordes a las necesidades de la Institución. Entre sus particularidades se encuentra la presencia de restricciones que deben ser satisfechas. El propósito de conocer la influencia estudiada en la aplicación del método de algoritmos genéticos fue logrado. Finalmente, se reconoce la automatización de una tarea así mismo la reducción del tiempo a unos cuantos segundos necesario para encontrar una solución, además, se encontraron indicios del impacto que la variación en cada uno de los parámetros tiene en el tiempo necesario para alcanzar una solución, lo cual ayudará a evaluar la conveniencia del uso de estas opciones de solución.*

Palabras clave: Algoritmo genético, evolutivo, generación, influencia.

Abstract: *This article demonstrates the convenience of applying a genetic algorithm to support the generation of academic schedules in order to offer an option to optimize the process for assigning teachers to the Ana Luisa Leoro Educational Unit, an educational institution with the highest size in number of students, teachers and infrastructure. The research method was mixed quantitative - qualitative where functions for the optimization of resources such as time, availability and accessibility to the information of schedules were analyzed; Through the interview, the precise data were collected to carry out an applied research with the support of the XP methodology with its 4 phases: (G1) planning, (G2) design, (G3) coding and (G4) testing, focusing on accelerating the process of solving large problems. The solution was modeled and the proposed algorithm was implemented in the Python programming language with the IDE JetBrains PyCharm according to the needs of the Institution. Among its peculiarities is the presence of restrictions that must be satisfied. The purpose of knowing the influence studied in the application of the method of genetic algorithms was achieved. Finally, the automation of a task is recognized as well as the reduction of the time to a few seconds necessary to find a solution, in addition, indications were found of the impact that the variation in each of the parameters has on the time necessary to reach a solution, which will help evaluate the appropriateness of using these solution options.*

Keywords: Genetic algorithm, evolutionary, generation, influence.

1 Introducción

En las Instituciones educativas la creación de horarios es un procedimiento administrativo esporádico que se desarrolla al inicio de cada periodo académico y muchas veces la información de años anteriores no puede ser utilizada debido al cambio de circunstancias o malla curricular.

Ante ello, (Yousef et al., 2016) sitúa la asignación de horarios como un problema que por medios tradicionales no puede ser resuelto en un tiempo razonable, por lo cual se requiere del uso de herramientas automáticas y sofisticadas como son los algoritmos heurísticos, el proceso no es simple ya que se analiza con profundidad si existe una progresión de elementos que se transforman en limitaciones que corresponden ser consideradas al momento de crear un horario académico.

Para la selección de los parámetros, (Hañari Mamani, 2016) refiere que el algoritmo evolutivo funciona en base a medidas heurísticas y no determinísticas, por lo cual, es muy importante seleccionar los parámetros adecuados; sin embargo, no existe una metodología que indique los valores exactos que deben asignarse ya que varían de acuerdo con la naturaleza del problema.

De ahí que se localizan limitaciones físicas importantes por ejemplo que una sala no puede ser utilizada por más

de un curso a la vez, o que un docente no puede realizar más de una clase en el mismo momento; limitaciones reglamentadas, en la que un alumno no debe tener concurrencia de horario entre dos de sus cursos; o restricciones legales que un docente no puede trabajar más de cierta cantidad de períodos semanales inicio de año. La asignación de los horarios de clases, dependiendo del tamaño de la unidad educativa puede ser tan simple o complejo de llevar a cabo; tomando su resolución mucho tiempo, en algunos casos realizado en forma manual como un juego de rompecabezas o utilizando medios automáticos para obtener una solución o aproximación a una buena solución.

En los AG se establecen procesos genéticos de las entidades biológicas que representan el principio de selección natural, siendo posible con ellos desarrollar la forma de corregir dificultades del mundo real proponiendo aplicar un algoritmo genético para la generación de horarios con vistas a ofrecer una opción de procedimiento para la asignación automatizada.

Al respecto conviene decir que los algoritmos genéticos son ampliamente usados (Yousef et al., 2016), la importancia de su aplicación, radica en, según (Flores Pichardo, 2011), la rapidez del proceso y la calidad de la solución obtenida.

En este trabajo se presentan los principales resultados tomando un orden histórico y de construcción. En la segunda sección se muestra la teoría de dominio y trabajos previos. La tercera sección describe los conceptos de AG. Seguidamente, en la cuarta sección se muestra los experimentos y resultados. Y finalmente se presenta las conclusiones del estudio.

2 Teoría del dominio y trabajos previos

Para (Arias-Osorio et al., 2018) el tema deja ver una evolución con respecto a los modelos tradicionales, lineales, resueltos usualmente por métodos exactos. Si bien estos siguen siendo desarrollados, en los últimos años el modelamiento del problema no solo ha incluido una gran variedad de nuevas restricciones, también se ha enfocado en incluir estas restricciones de forma más sencilla en el modelo, para técnicas de solución como metaheurísticas e híbridos.

(Paspuel López & Naranjo Luzuriaga, 2017) reconoce las soluciones aleatorias de AG con el fin de poder reiniciar, así define si con estas pequeñas variaciones mejora o no la optimización en efecto a lo largo del proceso; los reinicios aleatorios siempre pueden conseguir mejores soluciones.

Con relación a los métodos que están implementado en varias funciones de los AG para la generación de horarios basados en Python según (Calle-López et al., 2018) es un tema complicado, pero a su vez cuenta con múltiples soluciones algorítmicas.

3 Algoritmos Genéticos

Los algoritmos genéticos (AG) según (Cano et al., 2018) funcionan entre el conjunto de soluciones de una población natural, codificando la información de cada solución en una cadena, generalmente binaria, llamada cromosoma. Cuando la representación de los cromosomas se hace con cadenas de dígitos binarios se le conoce como genotipo.

Los cromosomas evolucionan a través de iteraciones, llamadas generaciones. En cada generación, los cromosomas son evaluados usando alguna medida de aptitud. Las siguientes generaciones (nuevos cromosomas), son generadas aplicando los operadores genéticos repetidamente, siendo estos los operadores de selección, cruzamiento, mutación y reemplazo.

3.1 Métodos Heurístico

Según (Flores Pichardo, 2011) son procedimientos eficientes para encontrar buenas soluciones. En estos métodos, la rapidez del proceso es tan importante como la calidad de la solución obtenida.

Un método heurístico es un procedimiento para resolver un problema de optimización bien definido mediante una aproximación intuitiva, en la que la estructura del problema se utiliza de forma inteligente para obtener una buena solución

3.2 Métodos Metaheurísticos

El término según (Ramíres Rodríguez, 2006) fue introducido por Fred Glover en 1986 y a partir de entonces han aparecido muchas propuestas de modelos o metas para delinear principales procedimientos de solución de problemas combinatorios

Las metaheurísticas según (Flores Pichardo, 2011) sitúan conceptualmente “por encima” de las heurísticas en el sentido de que guían el diseño de éstas, pueden estar compuestas por una combinación de algunas heurísticas, por ejemplo, una metaheurística puede usar una heurística constructiva para generar una solución inicial y luego usar otra heurística de búsqueda para encontrar una mejor solución.

4 Experimentos y Resultados

Para el desarrollo se utilizó la metodología XP, comprendida en las siguientes fases:

4.1 G1. Planificación

Para la planificación se utilizó las herramientas informáticas y el software Pychar como intérprete de Python con la ayuda del personal administrativo y un técnico en el desarrollo del proceso de los algoritmos.

4.2 G2. Diseño

Mediante la entrevista realizada se dio a conocer que la institución no tiene ningún sistema de creación o forma automática para crear un horario académico, y se hizo referencia a la falta de conocimientos en nuevas tecnologías y creación, se realizaron reuniones con personal administrativo de la institución obteniendo datos importantes para la creación del algoritmo genético.

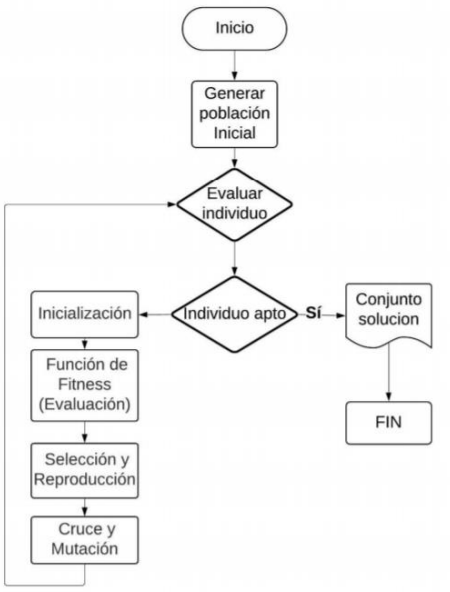


Figura 1: Diagrama de flujo del algoritmo genético usado en la generación de horarios

4.3 G3. Codificación

4.3.1 Inicialización

La población inicial puede generarse al azar, pero debe ser lo suficientemente grande y diversa para que durante la evaluación los individuos muestren un fitness distinto como la primera crea un individuo que después será guardado dentro de una matriz llamada "población", junto al resto de individuos:

```
def individual(min, max):
  return[random.randint(min, max) for i in range(largo)]
```

Su funcionamiento debería resultar evidente. Recibe como parámetros dos números enteros (un mínimo y un máximo) y se llena una lista de longitud dada 8 por la variable 'largo' con números aleatorios entre el mínimo y el máximo. Esta lista creada será el nuevo individuo. Para acabar, se crea una población inicial y el bucle del programa. El algoritmo hará evolucionar a la población durante cien generaciones, llamando las funciones que se han definido arriba:

```
population = crearPoblacion
print("Poblacion Inicial:\n%s"%(population))

for i in range(100):
  population = selection_and_reproduction(population)
  population = mutation(population)
   print("\nPoblacionFinal:\n%s"%(population))
   print("\n\n")
```

La siguiente función sirve para crear una población:

```
def crearPoblacion():
  return [individual(1,9) for i in range(num)]
```

Llamo a la función para crear individuales un número de veces igual a 'num', que definía el tamaño de la población. Todos estos nuevos individuales los guarda dentro de una lista, que devuelve fuera de la función.

```
def calcularFitness(individual):
 fitness = 0
 for i in range(len(individual)):
      if individual[i] == modelo[i]:
      fitness += 1
  return fitness
```

4.3.2 Función de Fitness (Evaluación)

La Función de Fitness evalúa cada uno de los individuos de la población y les asigna un valor numérico según su calidad. Este valor numérico se llama fitness.

```
def calcularFitness(individual):
 """
   Calcula el fitness de un individuo concreto.
"""
   fitness = 0
   for i in range(len(individual)):
       if individual[i] == modelo[i]:
       fitness += 1
    return fitness
```

La inicialización ya está hecha (es la función crearPoblación). Ahora se va a usar una función que se llama selection_and_reproduction() para evaluar cada uno de los individuos (evaluación), seleccionar los mejores (selección) y mezclar su material genético (crossover) a fin de crear una nueva población encima de la anterior.

```
def selection_and_reproduction(population):
  for i in range(len(population)-pressure):
     punto = random.randint(1,largo-1)
     padre = random.sample(selected, 2)
    population[i][:punto] = padre[0][:punto]
    population[i][punto:] = padre[1][punto:]
 return population
```

4.3.3 Selección y reproducción

Hay varias formas de seleccionar los individuos con el fitness más elevado para crear una nueva población, también es necesaria una función de mutación, que añada pequeñas variaciones al azar en el array de los individuos de la nueva generación.

```
def mutation(population):
   for i in range(len(population)-pressure):
      if random.random() <= mutation_chance:
         punto = random.randint(0,largo-1)
         nuevo_valor = random.randint(1,9)
         while nuevo_valor == population[i][punto]:
            nuevo_valor = random.randint(1,9)
       population[i][punto] = nuevo_valor
 return population
```

Cruce y Mutación

Una vez se han seleccionado los mejores individuos, el Cruce consiste en mezclar el material genético de estos para crear los nuevos individuos. La forma más sencilla de hacerlo es mediante un One-point Cruce. Consiste en elegir un punto al azar de los dos individuos e intercambiar el material genético a partir de esta posición. Para acabar, se crea una población 10 inicial y el bucle del programa. El algoritmo hará evolucionar a la población durante cien generaciones, llamando las funciones que se han definido arriba:

```
population = crearPoblacion()
print("Poblacion Inicial:\n%s"%(population)) #Se muestra la poblacion inicial
for i in range(100):
    population = selection_and_reproduction(population)
   population = mutation(population)
print("\nPoblacion Final:\n%s"%(population))
print("\n\n")
```

```
+-------------------------------------------------------------------------------+
| GENERACION DE    HORARIO     UNIDAD      EDUCATIVA      ANA       LUIS       LEORO          |
|-------------------------------------------------------------------------------|
+-------------------------------------------------------------------------------+
| Hora            LUNES        MARTES         MIERCOLES       JUEVES         VIERNES|
|-------------------------------------------------------------------------------|
| 12:30          MATEMATICA    EESS           MATEMATICA      INGLES         L. LITERATURA  |
| 13:30          MATEMATICA    EESS           MATEMATICA      INGLES         L. LITERATURA  |
| 14:30          CCNN          CCNN           EESS            MATEMATICA     CCNNN          |
| 15:30          RECESO        RECESO         RECESO          RECESO         RECESO         |
| 16:00          L. LITERATURA CCNN           EESS            MATEMATICA     EESS           |
| 17:00          L. LITERATURA ARTISTICA      E. FISICA       E. CULTURAL    INGLES         |
| 18:00          INGLES        ARTISTICA      E. FISICA       E. CULTURAL    INGLES         |
+-------------------------------------------------------------------------------+

Process finished with exit code 0
```

Figura 2: Generación de horarios

4.3.4 Pruebas

Para el análisis de resultados se consideran diferentes parámetros los cuales influyen de manera en el comportamiento de los AG y en el tiempo de obtención de los resultados. Entre los parámetros introducidos están:

El número de individuos que conforma la población

• La probabilidad de cruzamiento se utiliza con una posibilidad alta de acción sobre cada pareja de padres a cruzar (0.6 y 0.9), si no actúa los padres son los sucesores del proceso de mezcla de la pareja.

• El Operador de mutación en un punto seleccionado aleatoriamente, con previa probabilidad de mutación sobre cromosoma.

Los valores de entrada para un determinado de generaciones como número de docentes (D), número de materias (M), número de periodos académicos (P) y número de aulas (A) se toman para la evolución en similar de múltiples soluciones óptimas.

```
modelo = [1,1,1,1,1,1,1,1,1,1,1,1] #Objetivo a alcanzar
largo = 6
num = 7
pressure = 3
mutation_chance = 0.2
```

Figura 3: Valores de entrada

Todos los valores fueron extraídos los cuales se cargan en el prototipo, tomando parámetros fijos, como una población de 62 cromosomas que da como resultado el número de docentes de la institución se optó por este tamaño de muestra debido a la efectividad del algoritmo genético, también se considera una probabilidad de cruzamiento en un 0.95 y una probabilidad de mutación de 0.1, forma que exista una mayor cantidad de generaciones de predecesores. Por lo cual se observó la conveniencia de hacer variar los parámetros de entrada como se muestra en la siguiente figura.

```
Modelo: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Poblacion Inicial:
[[5, 3, 3, 6, 6, 3], [1, 6, 4, 5, 7, 5], [6, 1, 3, 3, 6, 2], [1, 7, 2, 5, 6, 6], [5, 5, 5, 2, 3, 6], [7, 1, 2, 3, 3, 7], [1, 6, 6, 5, 3, 7]]

Poblacion Final:
[[1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1]]
```

Figura 4: Prototipo algoritmo genético 1 – D, M, P y A

4.3.5 Análisis de la influencia

Como resultados se obtuvieron la reducción de tiempo en la generación de los horarios al utilizar los AG que trabajan con una población que deben pasar por los procesos de selección, combinación y mutación.

La influencia de los AG en la creación de horarios a comparación de algoritmos normales es muy diferente al ser más concretos ya que pueden crear dicho algoritmo que ayuden a ser más ágil por que los tiempos de ejecución son muy eficientes.

La creación manual de horarios académicos se realiza en la actualidad en la Unidad Educativa Ana Luisa Leoro con un tiempo estimado de 51 horas es decir que se demora mucho asignar a los docentes a sus aulas de clase.

Una medida que ayuda a conocer si las versiones secuenciales (CPU) y paralela (CPU+GPU) del algoritmo son comparables entre sí, es contar la cantidad de iteraciones realizadas del método principal y contar cuántas veces se computa la función de la una y en cada caso. Pueden afirmar que la cantidad de trabajo fue la misma y se pueden comparar.

Tabla 1: Tiempo de ejecución con algoritmo genético.

Puntos de mallado	Tiempo CPU	GPU Versión 1
10	0.045	0.135
22	0.156	0.263
56	0.598	0.467
86	1.478	1.745
154	2.975	2.471
197	6.138	3.864

4.3.6 Replica de AG

Tabla 2: Replica de ejecución con algoritmo genético.

Tiempo de ejecución (sg)		
Puntos de mallado	Tiempo CPU - Articulo	Tiempo CPU - Paper
10	0.045	0.000973701
22	0.156	0.000957727
56	0.598	0.002012014
86	1.478	0.002994299

154	2.975	0.005600929
197	6.138	0.012111187

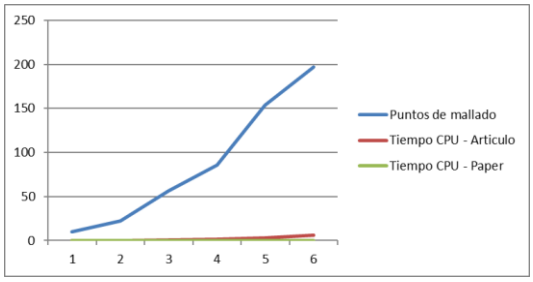


Figura 5: Gráfico de comparación AG

4.3.7 Complejidad

Complejidad del algortimo de la función de Fitness (evaluación) :

```
def calcularFitness(individual):
    """
    Calcula el fitness de un individuo concreto.
    """
    fitness = 0
    for i in range(len(individual)):
        if individual[i] == modelo[i]:
            fitness += 1
    return fitness
```

Operaciones primitivas:
Comparaciones del for: n
Comparaciones de la sentencia if: n-1
Por lo tanto:

costo $T_A(n) = n+n-1=2n-1 <=2n=c*n$, con $c>0$ constante

$T_A(n)$ ----> $O(n)$ o simplemente $T(n) = \boldsymbol{O(n)}$

5 Conclusiones

Con el desarrollo de este trabajo científico, se llega a la conclusión que se puede recomendar el uso del AG ya que paso por los procesos de selección, combinación y mutación para obtener los resultados.

Los resultados obtenidos al aplicar AG a un problema de optimización real, como lo es la asignación de profesores, han sido muy efectivo y eficiente. Por estas razones confirman las ventajas de usar un AG frente a otras técnicas, entre las conclusiones más interesantes puede distinguir la importancia de tener una población dinámica que logre manejarse rápidamente para no hacer más lenta de manera innecesaria la ejecución del algoritmo.

Esto ayudará con el tiempo a los encargados para que así realicen otras tareas de planeación académica y administrativa, permitiendo tener resultados rápidamente en caso de que requieran ser validados.

Referencias bibliográficas

Arias-Osorio, J., Karina Bautista, D., & Meneses-Pico, C. C. (2018). Revisión de literatura sobre los modelos de optimización en programación de turnos de enfermería. *Revista UIS Ingenierías*, *18*(2), 245–258. https://doi.org/10.18273/revuin.v18n2-2019023

Calle-López, D., Cornejo-Reyes, J., Pesántez-Avilés, F., Rodas-Tobar, M., Vásquez-Vásquez, C., & Robles-Bykbaev, V. (2018). Un sistema experto basado en minería de datos y programación entera lineal para soporte en la asignación de materias y diseño de horarios en educación superior. *Enfoque UTE*, *9*(1), 102–117. https://doi.org/10.29019/enfoqueute.v9n1.226

Cano, J. A., Correa-Espinal, A. A., & Gómez-Montoya, R. A. (2018). Solución del Problema de Conformación de Lotes en Almacenes utilizando Algoritmos Genéticos. *Información Tecnológica*, *29*(6), 235–244. https://doi.org/10.4067/s0718-07642018000600235

Flores Pichardo, M. (2011). Revisión de Algoritmos Genéticos Aplicados al Problema de la Programación de Cursos Universitarios. *Programación Matemática y Software*, *3*(1), 49–65. http://riaa.uaem.mx/handle/20.500.12055/93

Hañari Mamani, S. (2016). *Algoritmos evolutivos aplicados a la generación de horarios para el colegio aplicación de la UNA - Puno*. http://repositorio.unap.edu.pe/handle/UNAP/3008

Paspuel López, J. P., & Naranjo Luzuriaga, E. P. (2017). Análisis e implementación de un algoritmo de optimización para la asignación automática de aulas de clase en el campus sur de la Universidad Politécnica Salesiana. In *Universidad Politécnica Salesiana*.

Ramíres Rodríguez, C. O. (2006). Un algoritmo grasp con doble relajación para resolver el problema de floe shop schedulin. In *Pontificia Universidad Católica del Perú*.

Yousef, A. H., Salama, C., Jad, M. Y., El-Gafy, T., Matar, M., & Habashi, S. S. (2016). A GPU based genetic algorithm solution for the timetabling problem. *2016 11th International Conference on Computer Engineering & Systems (ICCES)*, 103–109. https://doi.org/10.1109/ICCES.2016.7821982